REVISION CHEAT SHEET:

CLASSES / OBJECTS

Classes, Objects, and Encapsulation in C#

Interview-ready notes · examples · C# · diagrams

Topic 11 of 40

This topic begins the **Object-Oriented Programming** portion of the roadmap.

Until now, most examples have focused on individual values and operations:

EXAMPLE / FLOW

```
Variables
    →
Methods
    →
Arrays
    →
Strings
```

Real applications need to model concepts that contain both:

EXAMPLE / FLOW

```
Data
+
Behavior
```

For the Fundraising Platform, examples include:

EXAMPLE / FLOW

```
Constituent

Gift

Campaign

Fund

GiftAllocation

Address

Interaction

Task
```

A `Gift`, for example, isn't merely:

EXAMPLE / FLOW

```
1000
```

It might contain:

EXAMPLE / FLOW

```
GiftId

TenantId
```

```
ConstituentId
CampaignId
GiftDate
TotalAmount
ReferenceNumber
Status
```


and also behavior such as:

EXAMPLE / FLOW

```
Allocate()
Cancel()
Refund()
ChangeCampaign()
Validate()
```

C# classes let us model these concepts.

This topic covers:

- object-oriented programming basics;
- class;
- object;
- instance;
- class vs object;
- state;
- behavior;
- identity;
- class members;
- fields;
- properties;
- methods;
- auto-implemented properties;
- backing fields;
- `get`;
- `set`;
- `private set`;
- get-only properties;
- `init`;
- `required`;
- object initialization;
- basic constructors;
- object initializers;
- `new`;
- `this`;
- access modifiers;

- `public`;
- `private`;
- `protected`;
- `internal`;
- `protected internal`;
- `private protected`;
- default accessibility;
- encapsulation;
- information hiding;
- maintaining invariants;
- exposing behavior instead of raw state;
- mutable vs controlled state;
- DTOs vs domain classes;
- class reference semantics;
- object identity;
- common mistakes;
- ASP.NET Core usage;
- Fundraising Platform modeling;
- performance considerations;
- security considerations;
- practical exercises;
- interview questions.

The central idea is:

EXAMPLE / FLOW

```
Class
    →
Blueprint / Type Definition
    →
Creates
    →
Objects / Instances
    →
Contain State
+
Expose Behavior
    →
Access Controlled Through Encapsulation
```

Microsoft describes a C# class as a **reference type that defines a blueprint for objects**. Objects created from that class are instances of the class.

What Is Object-Oriented Programming?

Object-oriented programming, usually called:

EXAMPLE / FLOW

```
OOP
```

is a programming style where software is organized around objects that represent concepts and responsibilities.

Instead of putting everything into unrelated variables:

```C#
long giftId = 1001;

decimal amount = 1000m;

bool cancelled = false;

string currency = "USD";
```

we can group related information into a meaningful type:

```C#
public class Gift
{
    public long GiftId
    {
        get;
        init;
    }

    public decimal TotalAmount
    {
        get;
        private set;
    }

    public string CurrencyCode
    {
        get;
        init;
    } = "USD";

    public bool IsCancelled
    {
        get;
        private set;
    }
}
```

Now the information belongs together.

Conceptually:

```EXAMPLE / FLOW
Gift
    →
GiftId
TotalAmount
CurrencyCode
IsCancelled
```

Later we can also add behavior:

```EXAMPLE / FLOW
Gift
    →
Cancel()
ChangeAmount()
Allocate()
```

The Four Main OOP Principles

You will study these individually throughout the next several topics:

EXAMPLE / FLOW

```
Encapsulation

Abstraction

Inheritance

Polymorphism
```

This topic focuses primarily on:

EXAMPLE / FLOW

```
Encapsulation
```

while introducing the class/object foundation required for all four.

What Is a Class?

Definition

A **class is a user-defined reference type that describes the data and behavior that its objects can have**.

Example:

C#

```
public class Gift
{
}
```

This creates a type called:

EXAMPLE / FLOW

```
Gift
```

It does not create an actual gift object yet.

Think of a class as:

EXAMPLE / FLOW

```
Blueprint

Template

Type Definition
```

Class Example

C#

```
public class Campaign
{
    public string Name
    {
        get;
        set;
```

```
    } = string.Empty;
}
```

CLASSES / OBJECTS

This says:

> Objects created from `Campaign` can have a `Name`.

The class describes the shape and capabilities of campaign objects.

Microsoft's current class documentation describes classes as reference types used to model objects that have state and behavior.

What Is an Object?

Definition

An **object is a runtime instance created from a type such as a class**.

Example:

C#

```
Campaign campaign = new Campaign();
```

Here:

EXAMPLE / FLOW

```
Campaign
    →
Class / Type

campaign
    →
Variable

new Campaign()
    →
New Object / Instance
```

Class vs Object

Class:

EXAMPLE / FLOW

```
Gift
```

describes what a gift object looks like.

Object:

C#

```
new Gift()
```

is an actual runtime instance.

Simple Analogy

EXAMPLE / FLOW

```
Architectural Blueprint
    →
Class

Actual House
    →
Object
```

One blueprint can create:

EXAMPLE / FLOW

```
House 1
House 2
House 3
```

Likewise:

EXAMPLE / FLOW

```
Gift Class
    →
Gift Object 1
Gift Object 2
Gift Object 3
```

Instance

The terms:

EXAMPLE / FLOW

```
Object
```

and:

EXAMPLE / FLOW

```
Instance
```

are often used almost interchangeably.

If:

C#

```
Gift gift = new Gift();
```

then:

EXAMPLE / FLOW

```
gift
```

refers to:

EXAMPLE / FLOW

```
an instance of Gift
```

One Class, Many Objects

Example:

```C#
Gift gift1 = new Gift();

Gift gift2 = new Gift();

Gift gift3 = new Gift();
```

Conceptually:

```EXAMPLE / FLOW
Gift Class
    →
Gift Object A

Gift Object B

Gift Object C
```

Each object can contain different state.

Object State

Definition

State is the data currently stored by an object.

Example:

```C#
public class Gift
{
    public decimal TotalAmount
    {
        get;
        set;
    }
}
```

Create:

```C#
Gift gift1 = new Gift
    {
        TotalAmount = 100m
    };

Gift gift2 = new Gift
    {
        TotalAmount = 500m
    };
```

State:

```EXAMPLE / FLOW
gift1
    →
TotalAmount = 100
```

```
gift2
    →
TotalAmount = 500
```

CLASSES / OBJECTS

They are objects of the same type but contain different state.

Object Behavior

Definition

Behavior describes operations an object can perform.

Example:

C#

```
public class Gift
{
    public decimal TotalAmount
    {
        get;
        private set;
    }

    public void IncreaseAmount(
        decimal amount)
    {
        TotalAmount += amount;
    }
}
```

Now:

EXAMPLE / FLOW

```
State
    →
TotalAmount

Behavior
    →
IncreaseAmount()
```

State + Behavior

Good object-oriented modeling often combines:

EXAMPLE / FLOW

```
Data
+
Rules governing that data
```

Example:

C#

```
public class Gift
{
    public decimal TotalAmount
    {
        get;
        private set;
    }
```

```
    public void ChangeAmount(
        decimal amount)
    {
        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        TotalAmount = amount;
    }
}
```

The object protects its own rule:

EXAMPLE / FLOW

```
Gift Amount
must be
greater than zero
```

What Is Object Identity?

Consider:

C#

```
Gift first = new Gift();

Gift second = new Gift();
```

Even if both have identical values, they are:

EXAMPLE / FLOW

```
two different objects
```

Conceptually:

EXAMPLE / FLOW

```
first
    →
Gift Object A

second
    →
Gift Object B
```

A `Gift` business entity usually has identity such as:

EXAMPLE / FLOW

```
GiftId
```

Two gifts both worth `$100` are still different gifts.

Class Is a Reference Type

Remember Topic 6:

EXAMPLE / FLOW

```
class
    →
reference type
```

Example:

C#

```
Gift first = new Gift();

Gift second = first;
```

This does not create another `Gift`.

Instead:

EXAMPLE / FLOW

```
first  ──┐
         ↓
      Gift Object
         ↑
second ──┘
```

Changing the object through one reference is visible through the other.

Microsoft explicitly documents that assigning one class variable to another copies the reference so both variables can refer to the same object.

Example

C#

```
Gift first = new Gift
    {
        TotalAmount = 1000m
    };

Gift second = first;

second.TotalAmount = 5000m;

Console.WriteLine(
    first.TotalAmount);
```

If `TotalAmount` has a public setter, result:

EXAMPLE / FLOW

```
5000
```

because there is one object.

Class Definition Syntax

Basic structure:

C#

```
public class Gift
{
}
```

Breakdown:

EXAMPLE / FLOW

```
public
    →
Accessibility

class
    →
Declares class type

Gift
    →
Type name

{ ... }
    →
Class body
```

Class Naming Convention

C# convention normally uses:

EXAMPLE / FLOW

```
PascalCase
```

for class names.

Good:

EXAMPLE / FLOW

```
Gift

GiftAllocation

FundraisingCampaign

ConstituentAddress
```

Avoid:

EXAMPLE / FLOW

```
gift

gift_class

giftallocation
```

for normal type names.

Class Members

A class can contain many kinds of members.

Important examples:

EXAMPLE / FLOW

```
Fields

Properties
```

```
Methods
✳
Constructors

Events

Nested Types

Operators

Indexers
```

CLASSES / OBJECTS

For now we focus primarily on:

EXAMPLE / FLOW

```
Fields

Properties

Methods

Basic Constructors
```

Example Class

C#

```
public class Gift
{
    private decimal _totalAmount;

    public decimal TotalAmount
    {
        get
        {
            return _totalAmount;
        }

        set
        {
            _totalAmount = value;
        }
    }

    public void PrintAmount()
    {
        Console.WriteLine(
            TotalAmount);
    }
}
```

Members:

EXAMPLE / FLOW

```
_totalAmount
    →
Field

TotalAmount
    →
Property

PrintAmount()
```

```
    →
Method
```

What Is a Field?

Definition

A **field is a variable declared directly inside a class or struct**.

Example:

```C#
public class Gift
{
    private decimal _totalAmount;
}
```

`_totalAmount` is a field.

Microsoft describes fields as variables that hold state for a type or object.

Field Example

```C#
public class Campaign
{
    private string _name = string.Empty;
}
```

The field belongs to each instance of:

```EXAMPLE / FLOW
Campaign
```

unless it is `static`.

Private Field Naming Convention

Common .NET convention:

```EXAMPLE / FLOW
_privateField
```

Example:

```C#
private decimal _totalAmount;

private string _currencyCode = "USD";

private bool _isCancelled;
```

This helps visually distinguish fields from properties and local variables.

Public Fields Are Usually Discouraged

This compiles:

```C#
public class Gift
{
    public decimal TotalAmount;
}
```

Then any caller can write:

```C#
gift.TotalAmount = -999999m;
```

The object has no opportunity to enforce its rules.

That weakens encapsulation.

Better

Use:

```C#
public decimal TotalAmount
{
    get;
    private set;
}
```

and expose behavior:

```C#
public void ChangeAmount(
    decimal newAmount)
{
    if (newAmount <= 0m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(newAmount));
    }

    TotalAmount = newAmount;
}
```

Now the caller cannot simply write:

```C#
gift.TotalAmount = -999m;
```

What Is a Property?

Definition

A **property provides controlled access to data using field-like syntax while executing accessor logic**.

Example:

```C#
public decimal TotalAmount
{
    get;
```

```
    set;
}
```


Caller:

```C#
decimal amount = gift.TotalAmount;
```

and:

```C#
gift.TotalAmount = 1000m;
```

looks similar to field access.

But a property is not simply a field.

Its access is controlled by:

```EXAMPLE / FLOW
get

set

init
```

accessors.

Microsoft describes properties as members that appear field-like to callers but execute `get`, `set`, or `init` accessor code.

Field vs Property

Field:

```C#
public decimal TotalAmount;
```

Property:

```C#
public decimal TotalAmount
{
    get;
    set;
}
```

Both may look similar to callers:

```C#
gift.TotalAmount
```

but properties provide an abstraction boundary.

Why Properties Are Preferred

A property can later include:

EXAMPLE / FLOW

```
Validation

Transformation

Different read/write accessibility

Calculated values

Initialization restrictions
```

without forcing callers to use a completely different syntax.

Full Property with Backing Field

Example:

C#

```
public class Gift
{
    private decimal _totalAmount;

    public decimal TotalAmount
    {
        get
        {
            return _totalAmount;
        }

        set
        {
            if (value <= 0m)
            {
                throw new ArgumentOutOfRangeException(
                    nameof(value));
            }

            _totalAmount = value;
        }
    }
}
```

Here:

EXAMPLE / FLOW

```
_totalAmount
    →
Backing Field

TotalAmount
    →
Property
```

What Is a Backing Field?

A backing field stores the data used by a property.

Flow:

EXAMPLE / FLOW

```
Caller
    →
gift.TotalAmount = 1000
    →
Property set accessor
    →
_validation_
    →
_totalAmount = 1000
```

Reading:

EXAMPLE / FLOW

```
Caller
    →
gift.TotalAmount
    →
get accessor
    →
return _totalAmount
```

get Accessor

The:

EXAMPLE / FLOW

```
get
```

accessor runs when a property is read.

Example:

C#

```
public decimal TotalAmount
{
    get
    {
        return _totalAmount;
    }
}
```

Call:

C#

```
decimal amount = gift.TotalAmount;
```

Flow:

EXAMPLE / FLOW

```
Read Property
    →
get executes
    →
Returns backing value
```

set Accessor

The:

accessor runs when a property is assigned after object creation or during initialization, depending on its accessibility.

Example:

```C#
public decimal TotalAmount
{
    get
    {
        return _totalAmount;
    }

    set
    {
        _totalAmount = value;
    }
}
```

The value Keyword

Inside a traditional `set` accessor:

```C#
set
{
    _totalAmount = value;
}
```

`value` means:

```
EXAMPLE / FLOW
the value caller is attempting to assign
```

Example:

```C#
gift.TotalAmount = 1000m;
```

Inside setter:

```
EXAMPLE / FLOW
value = 1000
```

Property Validation

```C#
public decimal TotalAmount
{
    get
    {
        return _totalAmount;
    }

    private set
```

```
    {
        if (value <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(value),
                "Gift amount must be greater than zero.");
        }

        _totalAmount = value;
    }
}
```

Now invalid values can be blocked.

Auto-Implemented Properties

For simple storage, you do not need to manually create a backing field.

Instead:

C#

```
public decimal TotalAmount
{
    get;
    set;
}
```

This is called an:

EXAMPLE / FLOW

```
auto-implemented property
```

or:

EXAMPLE / FLOW

```
auto-property
```

The compiler creates hidden storage automatically.

Auto-Property Mental Model

You write:

C#

```
public decimal Amount
{
    get;
    set;
}
```

Conceptually compiler provides storage:

EXAMPLE / FLOW

```
Hidden Backing Storage
    ↕
Amount Property
```

You normally don't access the compiler-generated backing field directly.

Auto-Property Example

C#
```
public class Campaign
{
    public string Name
    {
        get;
        set;
    } = string.Empty;

    public decimal GoalAmount
    {
        get;
        set;
    }
}
```

Simple and readable.

Property Initializer

A property can have a default initializer:

C#
```
public string CurrencyCode
{
    get;
    set;
} = "USD";
```

New objects receive:

EXAMPLE / FLOW
```
USD
```

unless initialization changes it.

Read-Write Property

C#
```
public string Name
{
    get;
    set;
}
```

Caller can:

EXAMPLE / FLOW
```
Read
+
Write
```

Read-Only to Caller with private set

```C#
public decimal TotalAmount
{
    get;
    private set;
}
```

Outside callers can:

EXAMPLE / FLOW

```
Read
```

but cannot directly:

EXAMPLE / FLOW

```
Write
```

The containing class can still assign it.

private set Example

```C#
public class Gift
{
    public decimal TotalAmount
    {
        get;
        private set;
    }

    public void ChangeAmount(
        decimal amount)
    {
        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        TotalAmount = amount;
    }
}
```

Caller:

```C#
gift.ChangeAmount(
    1500m);
```

Caller cannot:

```C#
// gift.TotalAmount = -500m;
```

This is strong encapsulation.

get-Only Property

Example:

```C#
public long GiftId
{
    get;
}
```

The property can be read externally.

It can be assigned from allowed initialization contexts such as a constructor.

Example:

```C#
public class Gift
{
    public Gift(
        long giftId)
    {
        GiftId = giftId;
    }

    public long GiftId
    {
        get;
    }
}
```

After construction:

```EXAMPLE / FLOW
GiftId
```

cannot normally be reassigned.

Microsoft documents get-only automatically implemented properties as assignable from the type's constructors.

init Accessor

Definition

`init` permits assignment during object initialization but prevents ordinary reassignment afterward.

Example:

```C#
public string CurrencyCode
{
    get;
    init;
} = "USD";
```

Create:

```C#
Gift gift = new Gift
    {
```

```
    CurrencyCode = "EUR"
};
```

CLASSES / OBJECTS

Allowed.

Later:

```
C#
// gift.CurrencyCode = "GBP";
```

is not allowed from normal caller code.

Microsoft documents `init` as an accessor intended for initialization-time assignment.

set vs init

`set`:

```
EXAMPLE / FLOW
Can normally assign
after object creation
```

`init`:

```
EXAMPLE / FLOW
Can assign
during initialization
but not ordinary later mutation
```

Example

```
C#
public sealed class Gift
{
    public long GiftId
    {
        get;
        init;
    }

    public decimal TotalAmount
    {
        get;
        private set;
    }

    public string CurrencyCode
    {
        get;
        init;
    } = "USD";
}
```

Interpretation:

```
EXAMPLE / FLOW
GiftId
    →
Set when object created
then stable
```

```
CurrencyCode
    →
Set when object created
then stable

TotalAmount
    →
Only Gift itself can change it later
```

When init Is Useful

Use it when data:

```
EXAMPLE / FLOW
Should be supplied during creation
```

but:

```
EXAMPLE / FLOW
Should not change freely later.
```

Typical examples:

```
EXAMPLE / FLOW
IDs

Creation metadata

Initial configuration

DTO properties

Value-like state
```

depending on the domain.

init Is Not Full Immutability

Suppose:

```
C#
public List<long> GiftIds
{
    get;
    init;
} = [];
```

After initialization, caller cannot replace:

```
EXAMPLE / FLOW
GiftIds
```

through the `init` accessor.

But the `List<long>` itself remains mutable:

```
C#
gift.GiftIds.Add(
    1001);
```

So:

EXAMPLE / FLOW

```
init property
≠
deeply immutable object
```

required Modifier

Definition

`required` tells the compiler that the member must be initialized when an instance is created.

Example:

```
C#
public sealed class Campaign
{
    public required string Name
    {
        get;
        init;
    }
}
```

Valid:

```
C#
Campaign campaign = new Campaign
    {
        Name = "Annual Giving"
    };
```

Invalid:

```
C#
// Campaign campaign = //     new Campaign();
```

because `Name` is required.

Microsoft documents `required` as requiring the member to be initialized when creating an instance. It was introduced in C# 11.

Why required Exists

Without `required`:

```
C#
public string Name
{
    get;
    init;
} = string.Empty;
```

you may accidentally create:

C#

```
Campaign campaign = new Campaign();
```

CLASSES / OBJECTS

even though a meaningful campaign must have a name.

`required` lets the type communicate:

EXAMPLE / FLOW

```
This value must be provided
during initialization.
```

required + init

One common pattern:

C#

```
public required string Name
{
    get;
    init;
}
```

Meaning:

EXAMPLE / FLOW

```
Caller must provide Name
+
Name cannot normally be changed later
```

This is especially useful for initialization-focused models.

required + set

`required` does not require `init`.

This is valid:

C#

```
public required string Name
{
    get;
    set;
}
```

Caller must initialize it, but it can later be reassigned.

So:

EXAMPLE / FLOW

```
required
```

and:

EXAMPLE / FLOW

```
init
```

solve different problems.

required Means Initialization Contract

`required` means:

EXAMPLE / FLOW

```
Must be initialized
```

It does not mean:

EXAMPLE / FLOW

```
Must always contain valid business data
```

For example:

C#

```
public required string Name
{
    get;
    init;
}
```

could still be initialized as:

C#

```
Name = "";
```

or possibly:

C#

```
Name = "   ";
```

if no additional validation exists.

Therefore:

EXAMPLE / FLOW

```
required
≠
business validation
```

required and Nullable References

Bad conceptual design:

C#

```
public required string? Name
{
    get;
    init;
}
```

This means:

EXAMPLE / FLOW

```
Caller must explicitly initialize Name
```

but the provided value can still be:

EXAMPLE / FLOW

```
null
```

That can be valid in rare cases, but usually ask whether that contract makes sense.

required Fields

C# also allows required fields in eligible cases.

Example:

C#

```
public required string Name;
```

But public fields generally provide weaker encapsulation.

Properties are normally preferred in application/domain models.

required Limitation

A `required` member must be accessible enough that callers who are allowed to create the type can satisfy the requirement. The compiler enforces rules around required-member accessibility.

required and Constructors

A constructor can satisfy required members.

The details include:

EXAMPLE / FLOW

```
SetsRequiredMembers
```

and constructor contracts.

That belongs mainly to Topic 12.

For this topic remember:

EXAMPLE / FLOW

```
required
    →
object creation must satisfy
required-member contract
```

Object Creation with new

To create a class instance:

C#

```
Gift gift = new Gift();
```

The:

EXAMPLE / FLOW

```
new
```

operator requests construction of a new object.

Microsoft documents class objects as being instantiated using `new`.

Target-Typed new

When compiler already knows the type:

```
C#
Gift gift = new();
```

is equivalent to:

```
C#
Gift gift = new Gift();
```

Use whichever is clearer in context.

var with new

```
C#
var gift = new Gift();
```

Compiler infers:

```
EXAMPLE / FLOW
Gift
```

The variable remains strongly typed.

This is not `dynamic`.

Basic Constructor Introduction

A constructor runs when an object is created.

Example:

```
C#
public class Gift
{
    public Gift(
        decimal amount)
    {
        TotalAmount = amount;
    }

    public decimal TotalAmount
    {
        get;
    }
}
```

Create:

```
C#
Gift gift = new Gift(
        1000m);
```

Flow:

EXAMPLE / FLOW

```
new Gift(1000)
    →
Allocate / create object
    →
Run constructor
    →
TotalAmount = 1000
    →
Return object reference
```

Microsoft documents constructors as initialization members invoked when objects are created using `new`.

Topic 12 will cover constructors in depth.

Why Constructors Are Useful

A constructor can make invalid incomplete objects harder to create.

Compare:

C#

```
Gift gift = new Gift();

gift.TotalAmount = 1000m;
```

with:

C#

```
Gift gift = new Gift(
        1000m);
```

The second version can enforce:

EXAMPLE / FLOW

```
Every Gift
must begin with an amount.
```

Object Initializers

Definition

Object initializers allow properties/fields to be assigned as part of object creation.

Example:

C#

```
Campaign campaign = new Campaign
    {
        Name = "Annual Giving",

        GoalAmount = 1_000_000m
    };
```

Microsoft documents object initializers as syntax for setting accessible `set` or `init` members during object creation.

Object Initializer Flow

EXAMPLE / FLOW

```
new Campaign
    →
Construct object
    →
Name = ...
    →
GoalAmount = ...
    →
Return initialized object
```

Object initializer assignments happen after the constructor's initialization work for the instance.

Example with required

C#

```
public sealed class Campaign
{
    public required string Name
    {
        get;
        init;
    }

    public decimal GoalAmount
    {
        get;
        init;
    }
}
```

Create:

C#

```
Campaign campaign = new Campaign
    {
        Name = "Annual Giving",

        GoalAmount = 500_000m
    };
```

Constructor + Object Initializer

You can combine them.

C#

```
public sealed class Gift
{
    public Gift(
        long constituentId)
    {
        ConstituentId = constituentId;
    }

    public long ConstituentId
    {
        get;
    }
```

```
    public long? CampaignId
    {
        get;
        init;
    }

    public string? Notes
    {
        get;
        init;
    }
}
```

Usage:

C#

```
Gift gift = new Gift(
        constituentId: 5001)
    {
        CampaignId = 100,

        Notes = "Annual campaign gift"
    };
```

Required core state can come from constructor.

Optional initialization can use object initializer.

this

Inside an instance member, `this` refers to the current object.

Example:

C#

```
public class Gift
{
    private decimal _amount;

    public void ChangeAmount(
        decimal amount)
    {
        this._amount = amount;
    }
}
```

Here:

EXAMPLE / FLOW

```
this
    →
current Gift object
```

When this Is Often Needed

Suppose field and parameter have the same name:

C#

```
public class Gift
{
```

```
    private decimal amount;

    public Gift(
        decimal amount)
    {
        this.amount = amount;
    }
}
```

Without:

```
EXAMPLE / FLOW
this.
```

both names could refer to the parameter.

Using standard `_amount` private-field naming avoids this conflict:

```
C#
private decimal _amount;

public Gift(
    decimal amount)
{
    _amount = amount;
}
```

Encapsulation

Definition

Encapsulation means grouping state and behavior inside a type while controlling how outside code interacts with that state.

In simple language:

> ⓘ An object should protect its important internal data and expose controlled ways to use or change it.

Encapsulation is not merely:

```
EXAMPLE / FLOW
put everything inside a class
```

It is:

```
EXAMPLE / FLOW
Class
    →
Owns State
    →
Controls Access
    →
Enforces Rules
```

Bad Encapsulation

```C#
public class Gift
{
    public decimal TotalAmount;

    public bool IsCancelled;

    public decimal RefundedAmount;
}
```

Any caller can write:

```C#
gift.TotalAmount = -1000m;

gift.IsCancelled = true;

gift.RefundedAmount = 5_000_000m;
```

The `Gift` object does not protect its rules.

Better Encapsulation

```C#
public sealed class Gift
{
    public decimal TotalAmount
    {
        get;
        private set;
    }

    public bool IsCancelled
    {
        get;
        private set;
    }

    public decimal RefundedAmount
    {
        get;
        private set;
    }

    public void ChangeAmount(
        decimal newAmount)
    {
        if (IsCancelled)
        {
            throw new InvalidOperationException(
                "A cancelled gift cannot be changed.");
        }

        if (newAmount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(newAmount));
        }
```

```
        TotalAmount = newAmount;
    }

    public void Cancel()
    {
        if (RefundedAmount > 0m)
        {
            throw new InvalidOperationException(
                "A refunded gift cannot be cancelled this way.");
        }

        IsCancelled = true;
    }
}
```


Now the object controls changes.

Encapsulation Flow

Bad:

```
EXAMPLE / FLOW
Caller
    →
Directly changes internal state
    →
Object can become invalid
```

Better:

```
EXAMPLE / FLOW
Caller
    →
Calls meaningful operation
    →
Object validates business rules
    →
State changes only if valid
```

Information Hiding

Encapsulation and information hiding are related.

Information hiding means keeping implementation details private so callers depend only on the public contract.

Example:

```
C#
private decimal _totalAmount;
```

Caller does not need to know:

```
EXAMPLE / FLOW
how amount is stored internally
```

Caller only needs:

```
C#
gift.TotalAmount
```

or:

```C#
gift.ChangeAmount(
    500m);
```

Why Information Hiding Matters

Suppose today:

```C#
private decimal _amount;
```

stores the value.

Tomorrow, implementation changes to:

EXAMPLE / FLOW

```
Amount in cents

Currency object

Calculated value

Database-backed value
```

If callers depend only on a stable public API, internal implementation can change with less impact.

Encapsulation vs Information Hiding

A useful distinction:

EXAMPLE / FLOW

```
Encapsulation
    →
Bundle state + behavior
and control interaction

Information Hiding
    →
Hide implementation details
that callers shouldn't depend on
```

They often work together.

Encapsulation Is More Than private Fields

A weak explanation is:

> ⓘ Encapsulation means private fields and public properties.

That is incomplete.

Better:

> ⓘ Encapsulation means protecting an object's valid state and exposing intentional operations rather than allowing unrestricted external manipulation.

Domain Invariant

Definition

An **invariant** is a rule that should always remain true for a valid object.

Gift examples:

EXAMPLE / FLOW

```
TotalAmount > 0

RefundedAmount >= 0

RefundedAmount <= TotalAmount

CurrencyCode must exist

Cancelled gift cannot be processed again
```

Encapsulation helps preserve invariants.

Weak Design

C#

```
public decimal TotalAmount
{
    get;
    set;
}

public decimal RefundedAmount
{
    get;
    set;
}
```

Caller can create:

EXAMPLE / FLOW

```
TotalAmount = 100

RefundedAmount = 1000
```

which violates the business rule.

Stronger Design

C#

```
public sealed class Gift
{
    public decimal TotalAmount
    {
        get;
        private set;
    }

    public decimal RefundedAmount
    {
        get;
        private set;
```

```
    }

    public void Refund(
        decimal amount)
    {
        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        decimal remaining = TotalAmount
            -
            RefundedAmount;

        if (amount > remaining)
        {
            throw new InvalidOperationException(
                "Refund exceeds remaining gift amount.");
        }

        RefundedAmount += amount;
    }
}
```

Now:

EXAMPLE / FLOW

```
Gift
```

protects its invariant.

Tell the Object What to Do

Poor design often looks like:

C#

```
if (!gift.IsCancelled)
{
    gift.TotalAmount = newAmount;
}
```

Caller understands internal business rules.

Better:

C#

```
gift.ChangeAmount(
    newAmount);
```

The object decides whether the action is legal.

This moves business responsibility to the correct place.

Property vs Method for Behavior

Use a property primarily for:

EXAMPLE / FLOW

```
state-like information
```

Example:

```
C#
gift.TotalAmount
```

Use a method for:

EXAMPLE / FLOW

```
operation / action / transition
```

Example:

```
C#
gift.Cancel();
```

```
C#
gift.Refund(
    100m);
```

Avoid properties that unexpectedly perform major operations.

Calculated Property

A property does not always need stored state.

Example:

```
C#
public decimal RemainingAmount
{
    get
    {
        return TotalAmount
               -
               RefundedAmount;
    }
}
```

No backing field needed.

Caller:

```
C#
decimal remaining = gift.RemainingAmount;
```

Expression-Bodied Property

Short form:

```
C#
public decimal RemainingAmount
    =>
    TotalAmount
    -
    RefundedAmount;
```

This property calculates its value when read.

Why Not Store Derived Values?

Suppose:

EXAMPLE / FLOW
```
TotalAmount = 1000
RefundedAmount = 200
```

Remaining should be:

EXAMPLE / FLOW
```
800
```

If you separately store:

EXAMPLE / FLOW
```
RemainingAmount
```

you now must keep three values synchronized.

Better:

C#
```
public decimal RemainingAmount
    =>
    TotalAmount
    -
    RefundedAmount;
```

when the value is naturally derived.

Access Modifiers

Access modifiers control:

EXAMPLE / FLOW
```
Who can use
this type or member?
```

C# provides six main accessibility levels:

EXAMPLE / FLOW
```
public

private

protected

internal

protected internal

private protected
```

Microsoft's current documentation defines these six levels.

public

Meaning

EXAMPLE / FLOW

```
Accessible from any code
that can access the containing type.
```

Example:

C#

```
public class Gift
{
    public decimal TotalAmount
    {
        get;
        private set;
    }
}
```

Outside code can read:

C#

```
gift.TotalAmount;
```

public Does Not Override Containing Type Accessibility

Suppose:

C#

```
internal class Gift
{
    public decimal Amount
    {
        get;
        set;
    }
}
```

The member says:

EXAMPLE / FLOW

```
public
```

but the class itself is:

EXAMPLE / FLOW

```
internal
```

So code outside that assembly generally cannot use `Gift` simply because a member inside it is public.

The containing type constrains what is actually reachable.

private

Meaning

EXAMPLE / FLOW

```
Accessible only inside
the containing class or struct.
```

Example:

C#

```
public class Gift
{
    private decimal _totalAmount;
}
```

Outside:

C#

```
// gift._totalAmount
```

is not accessible.

Why private Is Important

Private members let the class hide implementation details.

Example:

C#

```
private decimal _totalAmount;
```

allows the class to expose:

C#

```
public decimal TotalAmount
{
    get
    {
        return _totalAmount;
    }
}
```

without exposing storage directly.

protected

Meaning

Accessible from:

EXAMPLE / FLOW

```
The containing class
+
derived classes
```

including derived classes in other assemblies when the type hierarchy is accessible.

Example:

```C#
public class Donation
{
    protected decimal Amount;
}
```

Later:

```C#
public class Gift :
    Donation
{
    public void PrintAmount()
    {
        Console.WriteLine(
            Amount);
    }
}
```

This is valid because:

```EXAMPLE / FLOW
Gift
```

derives from:

```EXAMPLE / FLOW
Donation
```

Inheritance is Topic 13, so we will not go deeper here.

internal

Meaning

Accessible only within:

```EXAMPLE / FLOW
the same assembly
```

An assembly is typically produced as:

```EXAMPLE / FLOW
.dll

or

.exe
```

Example:

```C#
internal class GiftInternalHelper
{
}
```

Another class in the same project/assembly can normally use it.

Another assembly cannot normally access it.

Microsoft defines `internal` as accessibility limited to the current assembly.

Example Architecture

Imagine:

```
EXAMPLE / FLOW
Fundraising.Domain.dll

Fundraising.Api.dll
```

If:

```
C#
internal class GiftRuleEngine
{
}
```

exists in:

```
EXAMPLE / FLOW
Fundraising.Domain.dll
```

then normal code in:

```
EXAMPLE / FLOW
Fundraising.Api.dll
```

cannot directly use it.

That can help keep implementation details private to an assembly.

protected internal

This modifier frequently confuses developers.

Meaning

Accessible if:

```
EXAMPLE / FLOW
Same Assembly
OR
Derived Class
```

The important word is:

```
EXAMPLE / FLOW
OR
```

Microsoft defines it as available either from the same assembly or from derived classes in another assembly.

protected internal Flow

```
EXAMPLE / FLOW
Same Assembly?
    →
Yes
```

```
    →
Accessible

OR

Derived Class?
    →
Yes
    →
Accessible
```

private protected

Another frequently confused modifier.

Meaning

Accessible if code is:

EXAMPLE / FLOW

```
Derived
AND
in the same assembly
```

The important word is:

EXAMPLE / FLOW

```
AND
```

Microsoft describes `private protected` as the most restrictive protected form: derived types must also be in the same assembly.

private protected Flow

EXAMPLE / FLOW

```
Derived Class?
    →
Yes

AND

Same Assembly?
    →
Yes

Then
    →
Accessible
```

protected internal vs private protected

Remember:

EXAMPLE / FLOW

```
protected internal
    →
OR
```

EXAMPLE / FLOW

```
private protected
    →
AND
```

This is one of the easiest ways to remember them.

Access Modifier Table

Modifier	Same Class	Same Assembly Non-Derived	Derived Same Assembly	Derived Other Assembly	Unrelated Other Assembly
`public`	Yes	Yes	Yes	Yes	Yes
`private`	Yes	No	No	No	No
`protected`	Yes	No	Yes	Yes	No
`internal`	Yes	Yes	Yes	No	No
`protected internal`	Yes	Yes	Yes	Yes	No
`private protected`	Yes	No	Yes	No	No

This follows Microsoft's accessibility table.

Default Accessibility — Top-Level Class

Example:

C#

```
class Gift
{
}
```

A class declared directly at namespace level without an access modifier defaults to:

EXAMPLE / FLOW

```
internal
```

Microsoft documents top-level classes and structs as `internal` by default.

Default Accessibility — Class Members

Example:

C#

```
public class Gift
{
    decimal _amount;

    void Process()
    {
    }
}
```

Those members default to:

```
private
```

inside a class.

Microsoft documents class/struct members as private by default unless otherwise specified.

Should You Rely on Default Accessibility?

This works:

```
C#
class Gift
{
}
```

But many teams prefer explicitly writing:

```
C#
internal class Gift
{
}
```

when accessibility is important.

For members:

```
C#
private decimal _amount;
```

is usually clearer than depending on implicit private accessibility.

Type Accessibility vs Member Accessibility

Type:

```
C#
public class Gift
```

Member:

```
C#
private decimal _amount;
```

These are separate decisions.

Ask:

```
EXAMPLE / FLOW
Who should know this type exists?
```

and separately:

```
EXAMPLE / FLOW
Who should access this member?
```

Information Hiding Through Accessibility

A good class may expose only a small public surface.

Example:

C#

```
public sealed class Gift
{
    private decimal _totalAmount;
    private decimal _refundedAmount;

    public decimal TotalAmount
        =>
        _totalAmount;

    public decimal RemainingAmount
        =>
        _totalAmount
        -
        _refundedAmount;

    public void Refund(
        decimal amount)
    {
        // validation and mutation
    }
}
```

Caller knows:

EXAMPLE / FLOW

```
TotalAmount

RemainingAmount

Refund()
```

Caller does not know:

EXAMPLE / FLOW

```
how values are stored

how refund calculations are implemented
```

That is good information hiding.

Do Not Expose Everything

Bad:

C#

```
public decimal TotalAmount
{
    get;
    set;
}

public decimal RefundedAmount
{
    get;
```

```
    set;
}

public bool IsCancelled
{
    get;
    set;
}
```

CLASSES / OBJECTS

This is essentially:

EXAMPLE / FLOW

```
public bag of mutable data
```

Sometimes that is acceptable for:

EXAMPLE / FLOW

```
simple DTO
```

but usually weak for a domain object that must enforce rules.

DTO vs Domain Object

This distinction becomes important in ASP.NET Core.

A request DTO might reasonably have initialization-focused properties:

C#

```
public sealed class CreateGiftRequest
{
    public required long ConstituentId
    {
        get;
        init;
    }

    public required decimal Amount
    {
        get;
        init;
    }

    public long? CampaignId
    {
        get;
        init;
    }
}
```

It mainly carries data.

A domain entity may use stronger encapsulation:

C#

```
public sealed class Gift
{
    public long GiftId
    {
        get;
    }

    public decimal TotalAmount
```

```
    {
        get;
        private set;
    }

    public bool IsCancelled
    {
        get;
        private set;
    }

    public void Cancel()
    {
        // enforce domain rules
    }
}
```

Different responsibilities justify different property styles.

Data Object vs Rich Domain Object

Data-focused:

```
EXAMPLE / FLOW
Properties
Properties
Properties
```

Rich domain object:

```
EXAMPLE / FLOW
State
+
Behavior
+
Rules
+
Invariants
```

Neither approach is universally correct.

Use the amount of behavior appropriate for your application.

Avoid Anemic Domain Modeling When Rules Matter

An anemic model might contain:

```
C#
public class Gift
{
    public decimal Amount
    {
        get;
        set;
    }

    public bool Cancelled
    {
        get;
        set;
```

```
    }
}
```


and all rules live elsewhere.

That can make it easy for callers to bypass rules.

If a rule truly belongs to the `Gift` concept, putting the behavior inside `Gift` can improve correctness.

But Do Not Force Everything Into Entities

Encapsulation doesn't mean:

EXAMPLE / FLOW

```
every operation must live
inside one huge Gift class
```

For example:

EXAMPLE / FLOW

```
SendReceiptEmail

CallPaymentGateway

SaveToDatabase
```

are usually external/service responsibilities, not intrinsic `Gift` state behavior.

Good object design asks:

EXAMPLE / FLOW

```
Which responsibility belongs here?
```

not:

EXAMPLE / FLOW

```
Can I technically put this method here?
```

Property Validation vs Domain Method

This:

C#

```
public decimal Amount
{
    get
    {
        return _amount;
    }

    set
    {
        if (value <= 0m)
        {
            throw new ArgumentOutOfRangeException();
        }

        _amount = value;
    }
}
```

can work.

But sometimes:

```
C#
gift.ChangeAmount(
    amount);
```

is clearer because changing the amount is a business operation.

Especially if rules include:

```
EXAMPLE / FLOW
Gift cannot be cancelled

Payment cannot be settled

Audit information must be recorded
```

A method can communicate the operation more clearly than a general setter.

Setters Should Not Hide Major Side Effects

Avoid:

```
C#
gift.Amount = 1000m;
```

causing hidden:

```
EXAMPLE / FLOW
database save

email

HTTP request

payment processing
```

Property setters should usually remain reasonably predictable.

Major operations belong in explicit methods/services.

Encapsulation and Collections

Suppose:

```
C#
public List<GiftAllocation> Allocations
{
    get;
    set;
} = [];
```

Caller can:

```
C#
gift.Allocations.Clear();
```

or replace entire collection.

That may violate rules.

Stronger Collection Encapsulation

A future design might be:

```
C#
private readonly List<GiftAllocation>
    _allocations = [];

public IReadOnlyList<GiftAllocation>
    Allocations
    =>
    _allocations;

public void AddAllocation(
    GiftAllocation allocation)
{
    ArgumentNullException.ThrowIfNull(
        allocation);

    _allocations.Add(
        allocation);
}
```

Now:

```
EXAMPLE / FLOW
Gift
```

controls how allocations enter its state.

Still Understand Read-Only Interface Limitations

`IReadOnlyList<T>` means callers receive a read-only interface.

It does not automatically prove the underlying list can never be changed by the owning object.

That is often exactly what you want:

```
EXAMPLE / FLOW
Caller cannot mutate directly

Gift can mutate through controlled methods
```

Object Initializer vs Encapsulation

Object initializer:

```
C#
new Gift
{
    Amount = 1000m
}
```

is convenient.

But if every property is publicly settable solely to enable this syntax, you may weaken your domain model.

Do not prioritize concise initialization over valid-state protection.

Encapsulation and `init`

`init` is useful when values should be provided at creation but then become stable.

Example:

```C#
public required string ReferenceNumber
{
    get;
    init;
}
```

That is stronger than:

```C#
public string ReferenceNumber
{
    get;
    set;
}
```

if reference number should not freely change later.

Encapsulation and `required`

`required` helps make incomplete construction harder.

Example:

```C#
public required string CurrencyCode
{
    get;
    init;
}
```

But:

```EXAMPLE / FLOW
required
```

alone does not prove:

```EXAMPLE / FLOW
valid ISO currency

not whitespace

allowed by business rules
```

Those rules need validation.

Basic Fundraising Entity Design

```C#
public sealed class Gift
{
    public required long GiftId
```

```
    {
        get;
        init;
    }

    public required long ConstituentId
    {
        get;
        init;
    }

    public long? CampaignId
    {
        get;
        private set;
    }

    public decimal TotalAmount
    {
        get;
        private set;
    }

    public bool IsCancelled
    {
        get;
        private set;
    }

    public void ChangeCampaign(
        long? campaignId)
    {
        if (IsCancelled)
        {
            throw new InvalidOperationException(
                "A cancelled gift cannot be changed.");
        }

        CampaignId = campaignId;
    }

    public void ChangeAmount(
        decimal amount)
    {
        if (IsCancelled)
        {
            throw new InvalidOperationException(
                "A cancelled gift cannot be changed.");
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        TotalAmount = amount;
    }

    public void Cancel()
    {
        if (IsCancelled)
        {
            return;
        }
```

```
        IsCancelled = true;
    }
}
```

Notice the design:

EXAMPLE / FLOW

```
Outside Code
    →
Can Read

GiftId
ConstituentId
CampaignId
TotalAmount
IsCancelled
```

But outside code cannot freely mutate:

EXAMPLE / FLOW

```
CampaignId
TotalAmount
IsCancelled
```

It must use:

EXAMPLE / FLOW

```
ChangeCampaign()

ChangeAmount()

Cancel()
```

Why sealed in This Example?

`sealed` prevents inheritance.

Inheritance is Topic 13, so do not treat `sealed` as required for every class.

Here it simply says:

EXAMPLE / FLOW

```
This example isn't designed
as an inheritance base type.
```

We will study `sealed` properly later.

Object Creation Example

C#

```
Gift gift = new Gift
    {
        GiftId = 1001,

        ConstituentId = 5001
    };
```

But our earlier example hasn't yet initialized a valid amount through the public API.

We might call:

```C#
gift.ChangeAmount(
    1000m);
```

This works but permits a temporary incomplete state.

A constructor can improve this:

```C#
public Gift(
    long giftId,
    long constituentId,
    decimal amount)
{
    if (amount <= 0m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(amount));
    }

    GiftId = giftId;

    ConstituentId = constituentId;

    TotalAmount = amount;
}
```

Then:

```C#
Gift gift = new Gift(
        giftId: 1001,
        constituentId: 5001,
        amount: 1000m);
```

This better enforces valid construction.

Topic 12 will cover how to design constructors properly.

Public API Surface

Every public member becomes part of the way other code interacts with your class.

Think of:

```EXAMPLE / FLOW
public
```

as a design decision.

Not merely:

```EXAMPLE / FLOW
make compiler let me access this
```

Ask:

```EXAMPLE / FLOW
Should callers depend on this member?
```

If not:

EXAMPLE / FLOW

```
keep it private/internal/protected
as appropriate
```

Principle of Least Exposure

A useful design rule:

> Expose only what callers actually need.

Start restrictive:

EXAMPLE / FLOW

```
private
```

and widen only when required.

This reduces:

EXAMPLE / FLOW

```
coupling

accidental misuse

future compatibility constraints
```

Encapsulation Improves Maintainability

Suppose callers access:

C#

```
gift.ChangeAmount(
    1000m);
```

Internally you can later add:

EXAMPLE / FLOW

```
validation

audit tracking

currency checks

business restrictions
```

without changing every caller.

If callers directly modify fields:

C#

```
gift.Amount = 1000m;
```

business rules become scattered.

Encapsulation Improves Testability

You can test:

```
C#
gift.ChangeAmount(
    -100m);
```

and verify the object rejects it.

You can test:

```
C#
gift.Cancel();
```

and verify:

```
EXAMPLE / FLOW
IsCancelled = true
```

Behavior becomes explicit.

Encapsulation Improves Debugging

If state can only change through:

```
EXAMPLE / FLOW
ChangeAmount()

Cancel()

Refund()
```

you have clear places to debug.

If every piece of code can directly assign:

```
EXAMPLE / FLOW
gift.Status

gift.Amount

gift.RefundedAmount
```

finding where invalid state originated becomes harder.

Encapsulation Helps Security

Suppose:

```
C#
public bool IsAdministrator
{
    get;
    set;
}
```

on a sensitive user object.

Any code holding the object could potentially write:


```C#
user.IsAdministrator = true;
```

if that setter is available.

A safer model may expose:

```EXAMPLE / FLOW
controlled authorization/role operations
```

rather than raw writable security state.

Access Modifiers Are Not a Security Boundary by Themselves

`private` is an important software-design boundary.

But don't treat:

```EXAMPLE / FLOW
private
```

as an encryption or authorization system.

Runtime reflection, serialization, unmanaged code, and other mechanisms can bypass ordinary source-level accessibility in some scenarios.

Security still requires:

```EXAMPLE / FLOW
Authentication

Authorization

Validation

Least privilege

Secure storage
```

Access modifiers support encapsulation; they do not replace security controls.

Classes and ASP.NET Core

Classes are everywhere in ASP.NET Core.

Examples:

```EXAMPLE / FLOW
Controllers

Services

Repositories

DTOs

Options
```

```
Middleware
*
Background Services

Entity Models
```

CLASSES / OBJECTS

Request DTO Example

C#

```
public sealed class CreateGiftRequest
{
    public required long ConstituentId
    {
        get;
        init;
    }

    public required decimal Amount
    {
        get;
        init;
    }

    public long? CampaignId
    {
        get;
        init;
    }

    public string? Notes
    {
        get;
        init;
    }
}
```

This is primarily:

EXAMPLE / FLOW

```
Data Contract
```

so initialization properties are reasonable.

Service Example

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository
        _giftRepository;

    public GiftService(
        IGiftRepository giftRepository)
    {
        _giftRepository = giftRepository;
    }

    public void CreateGift(
        Gift gift)
    {
        _giftRepository.Save(
```

```
            gift);
    }
}
```


The dependency is:

EXAMPLE / FLOW

```
private
```

because callers don't need direct access to the repository.

That is information hiding.

Why Service Dependencies Are Usually private readonly

C#

```
private readonly IGiftRepository
    _giftRepository;
```

means:

EXAMPLE / FLOW

```
private
    →
implementation dependency hidden

readonly
    →
service keeps same dependency reference
after construction
```

This is a very common professional C# pattern.

Repository Encapsulation

Service code should usually call:

C#

```
_repository.Save(
    gift);
```

rather than knowing:

EXAMPLE / FLOW

```
SQL connection details

table names

database commands
```

That is another form of encapsulation at an architectural level.

Later architecture topics will expand this.

Common Mistake — Class = Object

Incorrect:

EXAMPLE / FLOW

```
Gift class
=
Gift object
```

Correct:

EXAMPLE / FLOW

```
Class
    →
Type / Blueprint

Object
    →
Runtime instance
```

Common Mistake — Object Variable Contains Entire Object

For classes, the variable normally contains:

EXAMPLE / FLOW

```
reference
```

to the object.

Example:

C#

```
Gift gift = new Gift();
```

Conceptually:

EXAMPLE / FLOW

```
gift variable
    →
reference
    →
Gift object
```

Common Mistake — Public Fields Everywhere

Avoid:

C#

```
public decimal Amount;
```

for meaningful domain state.

Prefer controlled properties and methods.

Common Mistake — Property = Field

A property can look like field access:

C#

```
gift.Amount
```

but it is a member with accessors.

Properties can execute logic.

Fields are storage variables.

Common Mistake — Automatic Property Means No Encapsulation

This:

```
C#
public decimal Amount
{
    get;
    set;
}
```

technically uses a property but still exposes unrestricted mutation.

Encapsulation is about:

```
EXAMPLE / FLOW
control
```

not merely replacing field syntax with property syntax.

Common Mistake — private set Makes Everything Safe

Example:

```
C#
public decimal Amount
{
    get;
    private set;
}
```

is stronger than a public setter.

But the class itself can still assign invalid data.

Encapsulation helps centralize rules; developers must still implement those rules correctly.

Common Mistake — required Means Valid

```
C#
public required string Name
{
    get;
    init;
}
```

does not prevent:

```
C#
Name = "";
```

`required` enforces initialization, not all business validation.

Common Mistake — init Means Immutable Object

C#

```
public List<string> Tags
{
    get;
    init;
} = [];
```

The list can still mutate.

`init` limits reassignment of the property, not all internal state mutation.

Common Mistake — Everything public Because "Other Classes Need It"

Too much `public` creates tight coupling.

Ask whether another type truly needs:

EXAMPLE / FLOW

```
The member itself
```

or only:

EXAMPLE / FLOW

```
an operation/result
```

Expose intention, not internal machinery.

Common Mistake — Everything private

The opposite extreme also fails.

If useful behavior must be called by external code, it needs appropriate visibility.

Encapsulation does not mean:

EXAMPLE / FLOW

```
Hide everything
```

It means:

EXAMPLE / FLOW

```
Expose a deliberate public contract
and hide unnecessary implementation.
```

Common Mistake — Business Rules in Controller

Bad ASP.NET Core design:

C#

```
if (request.Amount > 0m
    &&
    !gift.IsCancelled
    &&
    request.Amount <= 1_000_000m)
{
```

```
    gift.Amount = request.Amount;
}
```


The controller now understands internal `Gift` rules.

Better:

```C#
gift.ChangeAmount(
    request.Amount);
```

The controller orchestrates.

The domain object enforces its rule.

Common Mistake — Giant Class

Encapsulation does not mean every related action should live inside one enormous class.

Bad:

```EXAMPLE / FLOW
Gift
    →
Validate
SaveToDatabase
SendEmail
CallPaymentGateway
CreatePDF
UploadBlob
GenerateReport
AuthorizeUser
RefreshToken
```

This violates separation of responsibilities.

Later SOLID will address this directly.

Common Mistake — Property Setter Doing Database Work

Avoid:

```C#
gift.Amount = 1000m;
```

silently causing:

```EXAMPLE / FLOW
database update
HTTP call
email
```

Keep property semantics predictable.

Common Mistake — protected internal Confusion

Remember:

EXAMPLE / FLOW

```
protected internal
=
protected OR internal
```

Accessible from:

EXAMPLE / FLOW

```
same assembly
OR
derived class elsewhere
```

Common Mistake — private protected Confusion

Remember:

EXAMPLE / FLOW

```
private protected
=
protected AND same assembly
```

This is more restrictive.

Performance Considerations

Classes are reference types.

Creating:

C#

```
new Gift()
```

normally creates a managed object that the garbage collector manages.

One normal business object allocation is not something to fear.

Do Not Turn Everything Into Struct for Performance

If `Gift` has:

EXAMPLE / FLOW

```
Identity

Mutable lifecycle

Relationships

Complex behavior
```

a class is usually semantically appropriate.

Do not choose a struct merely to avoid allocation.

Correct semantics come first.

Property Access Cost

Simple auto-properties:

have tiny overhead and are routinely optimized by the JIT.

Do not expose public fields just because you think a property getter is too expensive.

That is normally premature optimization.

Expensive Getters

A property getter should generally not unexpectedly perform:

EXAMPLE / FLOW

```
Database query

Network request

Large calculation

Blocking operation
```

because callers expect property access to be relatively straightforward.

If something is expensive, a method often communicates that more clearly.

Object Graph Size

Classes can reference other classes:

EXAMPLE / FLOW

```
Gift
    →
Constituent

Gift
    →
Campaign

Gift
    →
Allocations
```

Large object graphs increase memory usage.

Do not eagerly load/retain enormous graphs without need.

That becomes especially important with EF Core and caching.

Encapsulation Can Improve Performance Flexibility

If callers interact through:

C#

```
gift.GetTotal();
```

or a property rather than directly depending on internal storage, you can later change implementation without changing callers.

Information hiding gives room for optimization.

Security Considerations

Never expose secrets as publicly writable members:

```C#
public string ApiKey
{
    get;
    set;
}
```

without reason.

Use the narrowest practical exposure.

Avoid Exposing Mutable Security State

Instead of:

```C#
public bool IsAuthorized
{
    get;
    set;
}
```

prefer authorization decisions through dedicated security infrastructure.

Business objects should not make it easy for arbitrary code to grant privileges by setting a property.

Multi-Tenant Fundraising Example

Suppose every `Gift` belongs to a tenant.

Avoid:

```C#
public long TenantId
{
    get;
    set;
}
```

if changing tenant after creation should never happen.

Prefer:

```C#
public long TenantId
{
    get;
}
```

initialized during construction.

Why?

A gift moving from:

```EXAMPLE / FLOW
Tenant A
```

to:

EXAMPLE / FLOW

```
Tenant B
```

could become a serious data-isolation problem.

Types can help make dangerous state transitions impossible.

Practical Exercise — Basic Class

Create:

```C#
public class Campaign
{
    public string Name
    {
        get;
        set;
    } = string.Empty;

    public decimal GoalAmount
    {
        get;
        set;
    }
}
```

Instantiate:

```C#
Campaign campaign = new Campaign
    {
        Name = "Annual Giving",

        GoalAmount = 500_000m
    };
```

Practical Exercise — Protect Amount

Improve:

```C#
public class Gift
{
    public decimal Amount
    {
        get;
        set;
    }
}
```

to:

```C#
public class Gift
{
    public decimal Amount
    {
        get;
```

```
        private set;
    }

    public void ChangeAmount(
        decimal amount)
    {
        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        Amount = amount;
    }
}
```

Now invalid external assignment is prevented.

Practical Exercise — required

C#

```
public sealed class Fund
{
    public required string Name
    {
        get;
        init;
    }

    public required string Code
    {
        get;
        init;
    }
}
```

Valid:

C#

```
Fund fund = new Fund
    {
        Name = "General Fund",

        Code = "GENERAL"
    };
```

Practical Exercise — init

C#

```
public sealed class Constituent
{
    public long ConstituentId
    {
        get;
        init;
    }

    public required string DisplayName
    {
        get;
        init;
```

```
    }
}
```


After initialization:

```
// constituent.DisplayName = "...";
```

is not normally allowed.

Practical Exercise — Derived Property

```
public class Gift
{
    public decimal TotalAmount
    {
        get;
        private set;
    }

    public decimal RefundedAmount
    {
        get;
        private set;
    }

    public decimal RemainingAmount
        =>
        TotalAmount
        -
        RefundedAmount;
}
```

If:

```
TotalAmount = 1000
RefundedAmount = 250
```

then:

```
RemainingAmount = 750
```

No separate field is necessary.

Practical Exercise — Encapsulation Failure

Given:

```
public class Gift
{
    public decimal TotalAmount
    {
        get;
        set;
    }
```

```
    public decimal RefundedAmount
    {
        get;
        set;
    }
}
```

What's wrong with:

```C#
gift.TotalAmount = 100m;

gift.RefundedAmount = 1000m;
```

Answer:

```EXAMPLE / FLOW
Object enters invalid state
because callers control related values independently.
```

Fix by restricting setters and exposing controlled operations.

Practical Exercise — Access Modifier

Which should be used for a helper method needed only inside the class?

Usually:

```C#
private
```

Example:

```C#
private bool IsAmountValid(
    decimal amount)
{
    return amount > 0m;
}
```

Practical Exercise — internal

Suppose:

```EXAMPLE / FLOW
Fundraising.Domain.dll
```

contains an implementation helper that should be used only inside that assembly.

Use:

```C#
internal sealed class GiftRuleEvaluator
{
}
```

Practical Exercise — protected internal

Requirement:

EXAMPLE / FLOW

```
Accessible anywhere in same assembly
OR by derived classes in another assembly.
```

CLASSES / OBJECTS

Answer:

EXAMPLE / FLOW

```
protected internal
```

Practical Exercise — private protected

Requirement:

EXAMPLE / FLOW

```
Accessible only by the class
or derived classes
that are also in the same assembly.
```

Answer:

EXAMPLE / FLOW

```
private protected
```

Interview Questions

What is a class in C#?

A class is a user-defined reference type that acts as a blueprint for objects.

It can contain:

EXAMPLE / FLOW

```
Fields

Properties

Methods

Constructors

Events

Other members
```

Classes are commonly used when a type has identity, behavior, mutable state, or inheritance requirements.

What is an object?

An object is a runtime instance of a type such as a class.

Example:

C#

```
Gift gift = new Gift();
```

`Gift` is the class/type.

The newly created `Gift` is the object/instance.

What is the difference between class and object?

Class:

EXAMPLE / FLOW

```
Definition / Blueprint / Type
```

Object:

EXAMPLE / FLOW

```
Runtime instance of that type
```

One class can produce many objects.

What is an instance?

An instance is an actual object created from a type.

C#

```
new Campaign()
```

creates a `Campaign` instance.

Are classes value types or reference types?

Classes are reference types.

A class variable stores a reference to an object rather than copying the entire object value on assignment.

What happens when one class variable is assigned to another?

The reference is copied.

Example:

C#

```
Gift second = first;
```

Both variables can point to the same object.

What is object state?

State is the data currently contained by an object.

Example:

EXAMPLE / FLOW

```
Gift.TotalAmount

Gift.IsCancelled

Gift.CampaignId
```

What is object behavior?

Behavior is the set of operations an object provides.

Examples:

EXAMPLE / FLOW

```
Cancel()

Refund()

ChangeAmount()
```

CLASSES / OBJECTS

What is a field?

A field is a variable declared inside a class or struct and used as part of that type's state.

What is a property?

A property is a member that provides field-like access through accessors such as:

EXAMPLE / FLOW

```
get

set

init
```

Property access can execute logic instead of exposing storage directly.

What is the difference between a field and a property?

Field:

EXAMPLE / FLOW

```
Direct storage variable
```

Property:

EXAMPLE / FLOW

```
Controlled access abstraction
with get/set/init accessors
```

Properties are usually preferred for publicly exposed object state.

What is a backing field?

A backing field is a private field used by a property to store data.

Example:

C#

```
private decimal _amount;

public decimal Amount
{
    get => _amount;

    private set =>
        _amount = value;
}
```

What is an auto-implemented property?

A property where the compiler automatically supplies backing storage.

Example:


```C#
public decimal Amount
{
    get;
    set;
}
```

What does get do?

The `get` accessor executes when a property is read.

Example:

```C#
decimal amount = gift.Amount;
```

What does set do?

The `set` accessor executes when a property is assigned.

The assigned value is available inside the accessor through:

EXAMPLE / FLOW

```
value
```

What is private set?

Example:

```C#
public decimal Amount
{
    get;
    private set;
}
```

Outside code can read the property, but only the containing type can normally use the setter.

What is init?

`init` permits setting a property during initialization while preventing ordinary later assignment by callers.

What is the difference between set and init?

`set`:

EXAMPLE / FLOW

```
allows ordinary reassignment
```

`init`:

EXAMPLE / FLOW

```
restricts assignment to initialization contexts
```

Does init make the entire object immutable?

No.

If an `init` property refers to a mutable object such as a `List<T>`, that object's contents can still change.

What is required?

`required` indicates that callers creating an instance must initialize the member as part of satisfying the type's required-member contract.

Does required validate business data?

No.

This:

```C#
public required string Name
{
    get;
    init;
}
```

requires initialization, but doesn't by itself reject:

EXAMPLE / FLOW

```
""

"   "
```

Business validation is separate.

Can required and init be used together?

Yes.

Common:

```C#
public required string Name
{
    get;
    init;
}
```

This means callers must initialize the property and ordinary later reassignment is restricted.

What is an object initializer?

Syntax that assigns accessible properties or fields while creating an object.

Example:

```C#
var campaign = new Campaign
    {
        Name = "Annual Giving"
    };
```

Object initializers work with accessible `set` or `init` members.

What is a constructor?

A constructor initializes an object when it is created.

Example:


```C#
public Gift(
    decimal amount)
{
    Amount = amount;
}
```

Constructor design is studied fully in Topic 12.

What does new do?

For normal class creation:

```C#
new Gift()
```

creates a new class instance and triggers the appropriate initialization/constructor process.

What is this in C#?

`this` refers to the current object instance inside an instance member.

Example:

```C#
this.Amount
```

means the `Amount` member of the current object.

What is encapsulation?

Encapsulation means placing related state and behavior inside a type while controlling how external code can interact with the object's internals.

A strong encapsulated object:

```EXAMPLE / FLOW
protects state

enforces rules

hides implementation details

exposes intentional operations
```

Is encapsulation just private fields and public properties?

No.

That is only one implementation technique.

True encapsulation is about:

```EXAMPLE / FLOW
protecting valid state
+
controlling interactions
+
exposing intentional behavior
```

What is information hiding?

Information hiding means preventing callers from depending on internal implementation details.

Example:

EXAMPLE / FLOW

```
private backing field
```

can change internally without requiring callers to change if the public property contract stays the same.

What is an invariant?

An invariant is a rule that should remain true for every valid instance.

Example:

EXAMPLE / FLOW

```
Gift.TotalAmount > 0
```

Encapsulation helps ensure invalid state cannot be introduced freely.

Why shouldn't fields normally be public?

Public fields allow callers to modify raw internal state directly.

Properties and methods provide stronger control, validation, compatibility, and information hiding.

What are access modifiers?

Keywords that control where a type or member can be accessed.

Main accessibility levels:

EXAMPLE / FLOW

```
public

private

protected

internal

protected internal

private protected
```

What does public mean?

Accessible from any code that can access the containing type.

What does private mean?

Accessible only within the containing class or struct.

What does protected mean?

Accessible from the containing class and its derived classes.

What does internal mean?

Accessible only within the same assembly.

What does protected internal mean?

Accessible from:

EXAMPLE / FLOW

```
same assembly
OR
derived types
```

including eligible derived types in other assemblies.

What does private protected mean?

Accessible from:

EXAMPLE / FLOW

```
the containing class
or derived classes
AND
within the same assembly
```

What is the difference between protected internal and private protected?

Remember:

EXAMPLE / FLOW

```
protected internal
=
OR
```

EXAMPLE / FLOW

```
private protected
=
AND
```

What is the default accessibility of a top-level class?

If no modifier is specified, a class declared directly at namespace level is:

EXAMPLE / FLOW

```
internal
```

by default.

What is the default accessibility of a class member?

A class member without an explicit access modifier is:

EXAMPLE / FLOW

```
private
```

by default.

Can a public member inside an internal class make the class public?

No.

The containing type's accessibility still limits whether external code can use the member through that type.

Why is encapsulation important?

It improves:

EXAMPLE / FLOW

```
Correctness

Maintainability

Testability

Readability

Security

Ability to change implementation

Protection of business invariants
```

What is the difference between a DTO and a domain class?

DTO:

EXAMPLE / FLOW

```
Primarily transports data
```

Domain class:

EXAMPLE / FLOW

```
Represents business concept
and may own behavior/rules
```

A DTO can reasonably expose initialization properties.

A domain class may use stronger encapsulation and behavioral methods.

Should every property have a public setter?

No.

Ask:

EXAMPLE / FLOW

```
Should arbitrary caller code
be allowed to change this value?
```

If not, consider:

EXAMPLE / FLOW

```
private set

init

get-only property

domain method
```

depending on the requirement.

Why is private readonly common for injected dependencies?

Example:

```C#
private readonly IGiftRepository
    _repository;
```

`private` hides the implementation dependency.

`readonly` keeps the reference stable after construction.

This supports encapsulation and predictable service design.

Should property setters perform database calls?

Normally no.

Property access is expected to be relatively straightforward.

Major side effects are clearer as explicit methods/services.

What is a good encapsulated Gift design?

Rather than exposing:

```C#
public bool IsCancelled
{
    get;
    set;
}
```

expose:

```C#
public bool IsCancelled
{
    get;
    private set;
}

public void Cancel()
{
    // validate rules

    IsCancelled = true;
}
```

The business operation controls state transition.

Best Practices

Use classes when you need:

EXAMPLE / FLOW

```
Identity

Complex behavior

Mutable lifecycle
```

```
Reference semantics
*
Inheritance
```

CLASSES / OBJECTS +

Prefer properties over public fields for public object state.

Use:

EXAMPLE / FLOW

```
private
```

for implementation details.

Expose only what callers genuinely need.

Use:

C#

```
get;
private set;
```

when callers should read but not directly mutate state.

Use:

C#

```
get;
init;
```

when a value should be supplied during initialization and remain stable afterward.

Use:

C#

```
required
```

when object creation must initialize a member.

Do not mistake:

EXAMPLE / FLOW

```
required
```

for complete validation.

Do not mistake:

EXAMPLE / FLOW

```
init
```

for deep immutability.

Use explicit behavior such as:

EXAMPLE / FLOW

```
Cancel()

Refund()

ChangeAmount()
```

when state transitions have business meaning.

Protect domain invariants inside the class when the rule truly belongs to the object.

Avoid public mutable fields.

Avoid exposing mutable collections directly when callers shouldn't control them.

Keep expensive or surprising side effects out of property accessors.

Use constructors to establish required valid state when appropriate.

Keep constructor details for Topic 12 rather than forcing all creation rules into property setters.

Use access modifiers intentionally instead of making everything `public`.

Remember:

EXAMPLE / FLOW

```
protected internal
=
OR

private protected
=
AND
```

Use DTOs for transport.

Use domain classes for business behavior when appropriate.

Do not create giant classes containing database, HTTP, email, payment, logging, and authorization responsibilities merely because the data is related.

Quick Revision

Concept	Meaning
Class	Reference-type blueprint
Object	Runtime instance
Instance	Another name for object created from type
State	Data held by object
Behavior	Operations object performs
Identity	What makes an entity distinct
Field	Storage variable inside type
Property	Controlled field-like access member
Backing field	Field used to store property value
`get`	Read property
`set`	Assign property
`private set`	Only type can normally set
`init`	Assignment restricted to initialization
`required`	Member must be initialized during creation contract
Constructor	Initializes new object
Object initializer	Sets members during object creation

Concept	Meaning
`new`	Creates instance
`this`	Current object instance
Encapsulation	Protect state and control interactions
Information hiding	Hide implementation details
Invariant	Rule that valid object should preserve
`public`	Broadest accessibility
`private`	Containing type only
`protected`	Type + derived classes
`internal`	Same assembly
`protected internal`	Same assembly **OR** derived
`private protected`	Derived **AND** same assembly

Complete Fundraising Example

C#

```
public sealed class Gift
{
    private readonly List<GiftAllocation>
        _allocations = [];

    public Gift(
        long giftId,
        long tenantId,
        long constituentId,
        decimal totalAmount,
        string currencyCode)
    {
        if (giftId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(giftId));
        }

        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        if (constituentId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(constituentId));
        }

        if (totalAmount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(totalAmount));
        }

        ArgumentException.ThrowIfNullOrWhiteSpace(
```

```
            currencyCode);

        GiftId = giftId;

        TenantId = tenantId;

        ConstituentId = constituentId;

        TotalAmount = totalAmount;

        CurrencyCode = currencyCode
                .Trim()
                .ToUpperInvariant();
    }

    public long GiftId
    {
        get;
    }

    public long TenantId
    {
        get;
    }

    public long ConstituentId
    {
        get;
    }

    public long? CampaignId
    {
        get;
        private set;
    }

    public decimal TotalAmount
    {
        get;
        private set;
    }

    public decimal RefundedAmount
    {
        get;
        private set;
    }

    public string CurrencyCode
    {
        get;
    }

    public bool IsCancelled
    {
        get;
        private set;
    }

    public decimal RemainingAmount
        =>
        TotalAmount
        -
        RefundedAmount;
```

```
    public IReadOnlyList<GiftAllocation>
        Allocations
        =>
        _allocations;

    public void ChangeCampaign(
        long? campaignId)
    {
        EnsureGiftIsActive();

        CampaignId = campaignId;
    }

    public void ChangeAmount(
        decimal amount)
    {
        EnsureGiftIsActive();

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        if (amount < RefundedAmount)
        {
            throw new InvalidOperationException(
                "Gift amount cannot be less than the amount already refunded.");
        }

        TotalAmount = amount;
    }

    public void Refund(
        decimal amount)
    {
        EnsureGiftIsActive();

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        if (amount > RemainingAmount)
        {
            throw new InvalidOperationException(
                "Refund exceeds the remaining gift amount.");
        }

        RefundedAmount += amount;
    }

    public void AddAllocation(
        GiftAllocation allocation)
    {
        ArgumentNullException.ThrowIfNull(
            allocation);

        EnsureGiftIsActive();

        decimal newAllocationTotal = GetAllocatedAmount()
            +
            allocation.Amount;
```

```
        if (newAllocationTotal
            >
            TotalAmount)
        {
            throw new InvalidOperationException(
                "Total gift allocations cannot exceed the gift amount.");
        }

        _allocations.Add(
            allocation);
    }

    public void Cancel()
    {
        EnsureGiftIsActive();

        if (RefundedAmount > 0m)
        {
            throw new InvalidOperationException(
                "A partially refunded gift cannot be cancelled using this operation.");
        }

        IsCancelled = true;
    }

    private decimal GetAllocatedAmount()
    {
        decimal total = 0m;

        foreach (
            GiftAllocation allocation
            in _allocations)
        {
            total += allocation.Amount;
        }

        return total;
    }

    private void EnsureGiftIsActive()
    {
        if (IsCancelled)
        {
            throw new InvalidOperationException(
                "The gift is cancelled.");
        }
    }
}

public sealed class GiftAllocation
{
    public GiftAllocation(
        long fundId,
        decimal amount)
    {
        if (fundId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(fundId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
```

```
        }

        FundId = fundId;

        Amount = amount;
    }

    public long FundId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }
}
```

Usage:

C#

```
Gift gift = new Gift(
        giftId: 1001,
        tenantId: 1,
        constituentId: 5001,
        totalAmount: 1000m,
        currencyCode: "USD");

gift.ChangeCampaign(
    2001);

gift.AddAllocation(
    new GiftAllocation(
        fundId: 3001,
        amount: 600m));

gift.AddAllocation(
    new GiftAllocation(
        fundId: 3002,
        amount: 400m));

gift.Refund(
    100m);

Console.WriteLine(
    gift.TotalAmount);

Console.WriteLine(
    gift.RemainingAmount);

Console.WriteLine(
    gift.IsCancelled);
```

External code cannot directly write:

C#

```
// gift.TotalAmount = -1000m;

// gift.RefundedAmount = 999999m;

// gift.IsCancelled = true;

// gift.Allocations.Clear();
```

That is exactly the purpose of encapsulation.

Complete Encapsulation Flow

EXAMPLE / FLOW

```
Caller
    →
Creates Gift
    →
Constructor validates required state
    →
Gift becomes valid object
```

Then:

EXAMPLE / FLOW

```
Caller
    →
Wants to change amount
    →
ChangeAmount()
    →
Gift checks
        →
Cancelled?
        →
Amount valid?
        →
Refund invariant valid?
    →
State changes
```

Refund:

EXAMPLE / FLOW

```
Caller
    →
Refund(100)
    →
Gift checks
        →
Active?
        →
Amount > 0?
        →
Amount <= Remaining?
    →
RefundedAmount updated
```

Allocation:

EXAMPLE / FLOW

```
Caller
    →
AddAllocation()
    →
Gift checks
        →
Valid allocation?
        →
Total allocations <= Gift amount?
```

```
    →
Private collection changes
```

Outside code sees:

EXAMPLE / FLOW

```
Public Contract
```

but internal storage remains:

EXAMPLE / FLOW

```
Private Implementation
```

Class Design Decision Flow

EXAMPLE / FLOW

```
Need custom concept?
    →
Does it have identity / lifecycle / behavior?
    →
Yes
    →
Class likely appropriate
```

Then ask:

EXAMPLE / FLOW

```
What state does object own?
    →
Fields / Properties
```

Then:

EXAMPLE / FLOW

```
Who may read state?
    →
get accessibility
```

Then:

EXAMPLE / FLOW

```
Who may change state?
    →
set / private set / init / methods
```

Then:

EXAMPLE / FLOW

```
Which state is mandatory?
    →
Constructor / required members
```

Then:

EXAMPLE / FLOW

```
Which rules must always hold?
    →
Encapsulate invariants
```

CLASSES / OBJECTS

Then:

EXAMPLE / FLOW

```
What should callers know?
    →
Public API
```

and:

EXAMPLE / FLOW

```
What should callers NOT know?
    →
Private/internal implementation
```

Final Concept Flow

EXAMPLE / FLOW

```
Class
    →
Defines
    →
State + Behavior
```

EXAMPLE / FLOW

```
new
    →
Creates
    →
Object / Instance
```

EXAMPLE / FLOW

```
Object
    →
Reference Type
    →
Variable holds reference
```

State:

EXAMPLE / FLOW

```
Fields
    →
Internal storage
```

Controlled exposure:

EXAMPLE / FLOW

```
Properties
    →
get
set
init
```

Construction:

EXAMPLE / FLOW

```
Constructor
+
Object Initializer
+
required
```

Protection:

EXAMPLE / FLOW

```
Access Modifiers
    →
public
private
protected
internal
protected internal
private protected
```

Encapsulation:

EXAMPLE / FLOW

```
Private Internal State
    →
Public Intentional Operations
    →
Validation
    →
Invariant Protection
    →
Valid Object
```

The most important lesson from this topic is:

> **A class is not merely a container for variables. A well-designed class represents a concept, owns its state, exposes intentional behavior, hides unnecessary implementation details, and prevents callers from placing the object into invalid states.**

For your Fundraising Platform:

EXAMPLE / FLOW

```
Gift
```

should not merely be:

EXAMPLE / FLOW

```
a group of public properties.
```

It should eventually represent:

EXAMPLE / FLOW

```
Gift Identity
+
Gift State
+
Gift Rules
```

```
+
Valid Gift Operations
```

CLASSES / OBJECTS

That understanding is the foundation for the next topics on constructors, inheritance, polymorphism, abstraction, interfaces, composition, SOLID, dependency injection, domain modeling, and ASP.NET Core architecture.

Next roadmap topic: Topic 12 — Constructors and Object Initialization.